

Transport Reliability & Incident Analytics API

API Documentation Overview

1. System Overview

This document provides formal API documentation for the Transport Reliability & Incident Analytics API. The API enables structured management and analytical evaluation of public transport route performance using authenticated user-submitted delay incidents.

The system is implemented using FastAPI with PostgreSQL persistence and JWT-based authentication.

2. Base URLs

Local Development Environment: <http://127.0.0.1:8000>

Deployment: <https://transport-reliability-api.onrender.com>

Docs: <https://transport-reliability-api.onrender.com/docs>

All endpoints are relative to the base URL.

Interactive OpenAPI documentation is available at: /docs

3. Authentication Model

Authentication is implemented using JSON Web Tokens (JWT) with Bearer authorization.

Registration

POST /auth/register

Creates a new user account.

Login

POST /auth/login

Returns an access token:

```
{
  "access_token": "<jwt_token>",
  "token_type": "bearer"
}
```

Using the Token

Protected endpoints require the HTTP header:

Authorization: Bearer <access_token>

Protected write operations on routes, stations, and incidents require Bearer authentication.

Authentication is not required for user registration or login.

4. Core Data Entities

The system models five relational entities:

- Users
- Routes
- Stations
- UserIncidents
- RouteStations (junction table)

Relationships:

- A user may create multiple incidents.
- A route may have multiple incidents.
- A station may have multiple incidents.
- Routes and stations are linked through a many-to-many association.
- Users may only modify or delete incidents they created.

5. Core Functional Areas

The API supports two primary functional domains:

5.1 CRUD Operations

Full Create, Read, Update, Delete operations are implemented for:

- Routes
- Stations
- UserIncidents

All write operations are protected by JWT authentication.

5.2 Analytical Endpoints

The API provides aggregated performance metrics:

1. Reliability Per Route

Calculates the percentage of incidents with $\text{delay} \leq \text{threshold}$ (default 5 minutes).

2. Delay Distribution

Groups incidents by:

- Hour of day
- Weekday

3. Hotspot Stations

Ranks stations using: $\text{pain_index} = \text{incident_count} \times \text{average_delay}$

These endpoints return aggregated JSON responses suitable for further analysis or visualisation.

Authentication & Error Handling Specification

6. HTTP Status Code Model

The API uses standard HTTP status codes.

Code	Meaning
200	Successful request
201	Resource successfully created
204	Resource deleted successfully
400	Bad request (e.g., duplicate email registration)
401	Unauthorized: missing or invalid Bearer token
403	Forbidden: authenticated but not resource owner
404	Resource not found

422	Validation error (invalid request payload or parameters)
-----	--

7. Ownership Enforcement

For incident resources:

- Only the user who created the incident may update or delete it.
- If another authenticated user attempts modification, the API returns:

403 Forbidden

```
{
  "detail": "Not allowed"
}
```

This ensures proper access control and data integrity.

8. Validation Model

Request validation is performed using Pydantic schemas.

Examples of validation enforcement:

- delay_minutes must be ≥ 0
- Required fields must be present
- Query parameters must be within defined ranges

Invalid payloads return: 422 Validation Error

9. Documentation Structure

The following pages contain the generated OpenAPI (Swagger) documentation exported directly from the deployed API. This automatically generated specification includes endpoint definitions, parameters, request schemas, response models, and authentication requirements for all API operations.

Transport Reliability API

0.1.0 OAS 3.1

/openapi.json

Authorize 

auth

POST	/auth/register	Register	
POST	/auth/login	Login	

incidents

POST	/incidents	Create Incident	 
GET	/incidents/{incident_id}	Get Incident	
PATCH	/incidents/{incident_id}	Update Incident	 
DELETE	/incidents/{incident_id}	Delete Incident	 

routes

POST	/routes	Create Route	 
GET	/routes	List Routes	
GET	/routes/{route_id}	Get Route	
PATCH	/routes/{route_id}	Update Route	 
DELETE	/routes/{route_id}	Delete Route	 

stations



POST	/stations	Create Station		▼
GET	/stations	List Stations		▼
GET	/stations/{station_id}	Get Station		▼
PATCH	/stations/{station_id}	Update Station		▼
DELETE	/stations/{station_id}	Delete Station		▼

analytics



GET	/analytics/routes/{route_id}/reliability	Reliability Per Route		▼
GET	/analytics/delays/distribution	Delay Distribution		▼
GET	/analytics/stations/hotspots	Hotspot Stations		▼

root



GET	/	Root		▼
-----	---	------	--	---

Schemas



Body_login_auth_login_post > Expand all object

DelayBucketOut > Expand all object

HTTPValidationError > Expand all object

HotspotOut > Expand all object

HotspotsResponse > Expand all object

IncidentCreate > Expand all **object**

IncidentOut > Expand all **object**

IncidentUpdate > Expand all **object**

ReliabilityOut > Expand all **object**

RouteCreate > Expand all **object**

RouteOut > Expand all **object**

RouteUpdate > Expand all **object**

StationCreate > Expand all **object**

StationOut > Expand all **object**

StationUpdate > Expand all **object**

TokenOut > Expand all **object**

UserRegister > Expand all **object**

ValidationError > Expand all **object**